

School of Computers and Information Engineering

STUDENT LEARNING ACTIVITY

Fall Semester 2023

Data Structures (SOC 2010)

Dating service's lack of personalized recommendation system

Student Learning Assignment

Submitted by

U2210251, Javohirbek Xatamov(SEC-002)(CSE-22-08)

Index Page

Title	Page No 0
-------	-----------

- Problem Introduction
- Operations included in the application
- Choice of Data Structures
- Code
- limitations and Future Scope
- Subject Importance
- References (if any)

[Note – you can add more sections if you find any relevant to your work]



TOSHKENT SHAHRIDAGI INHA UNIVERSITETI
INHA UNIVERSITY IN TASHKENT

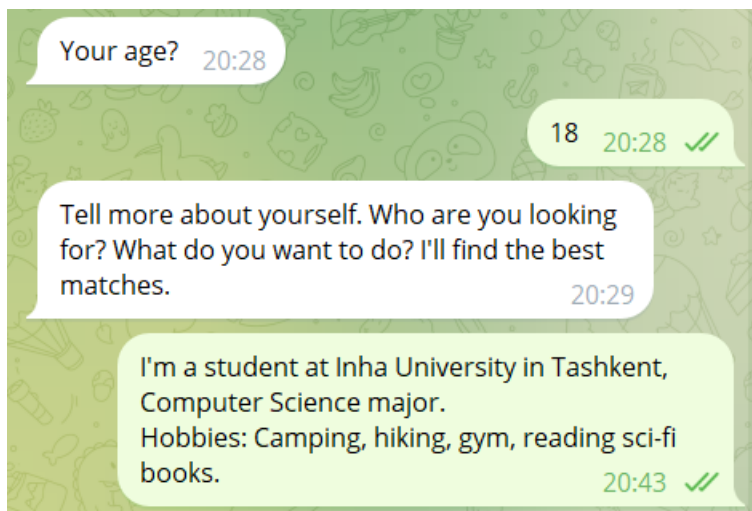
Problem Introduction

Problem Introduction	Page No 1
----------------------	-----------

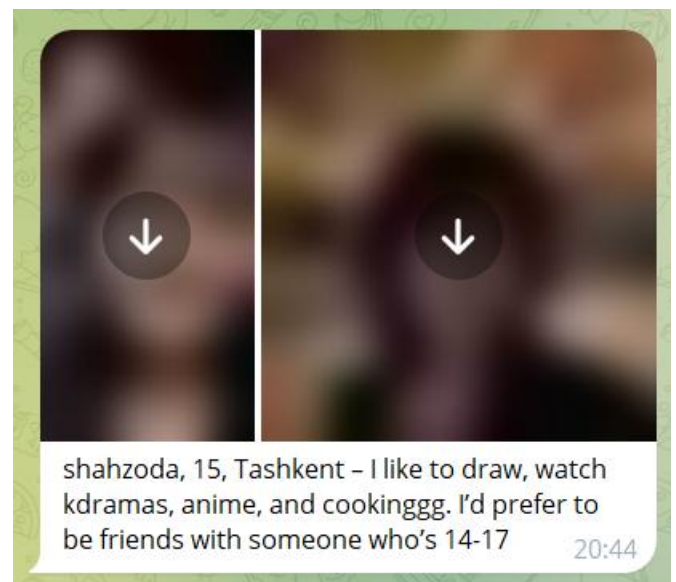
Recently new popular dating telegram bot named “Leo match” become popular among the population of Uzbekistan. It enables users to create an account and input the following information:

1. Language
2. Name
3. Age
4. Sex
5. City
6. Short Biography

After that, the service starts to recommend the user accounts of potential matches. But it doesn't take into account the interests of the user nor his preferred age difference, in fact, this service doesn't even have a function to create a “preference” for the recommendation algorithm. After some experiments, I came to a conclusion that this service only takes into account the age and city of the user for personalized recommendations , while completely ignoring the other information.



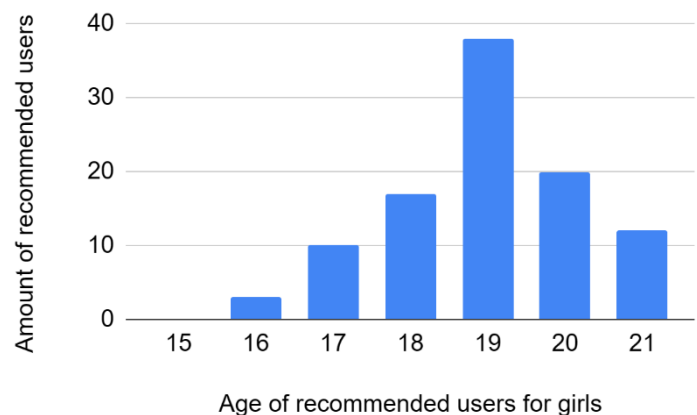
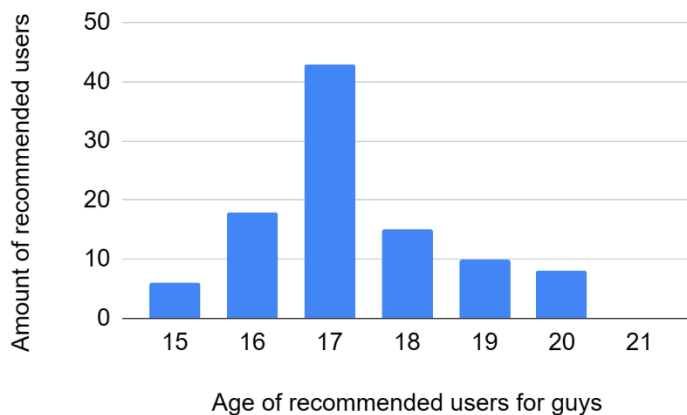
My bio in this Telegram Bot



One of the bot's suggestions

It can be clearly seen, that the bot ignored all the info about me and the girl in photo, as we don't have anything in common with this account beside living in the same city. After some experiments I also noticed that this service has a tendency to recommend guys, younger girls, and for girls, older guys. It probably does it due to the general tendency of men choosing younger women, and women choosing older men.

Problem statement: This service clearly lacks the broad system of personalized



Problem statement: This service clearly lacks the ability to suggest profiles based on the users "about info" like hobbies or the interests.

Operations included in the application

In order to improve the recommendation system I will need to change some of the UX features of the bot:

- 1) Users will need to choose their city from the given list, not just type it in manually, as it might cause them to use different alphabets or typos in the inputted names.
- 2) The same thing but with hobbies, we will have to cut the number of hobbies that can be listed, so the user can just select 2 hobbies from the list we will provide him with
- 3) We add a new parameter "preferred age" the age that the user prefers to date with.

Choice of Data Structures

In order to solve this problem I used several data types, but the main focus is on hash tables. They were chosen as integrating hash tables into the "Leo match" bot can help to improve its recommendation

system. In addition, hash tables provide a fast way to store and retrieve data, which can be useful in managing user preferences and matching algorithm. Using hash tables, we can add the recommendation algorithm to take user preferences into account. When suggesting matches, the preferences stored in the hash table will be considered and profiles that align with the user's specified criteria will be prioritized. Overall idea works like this:

Every user will have a unique account with these attributes chained to it

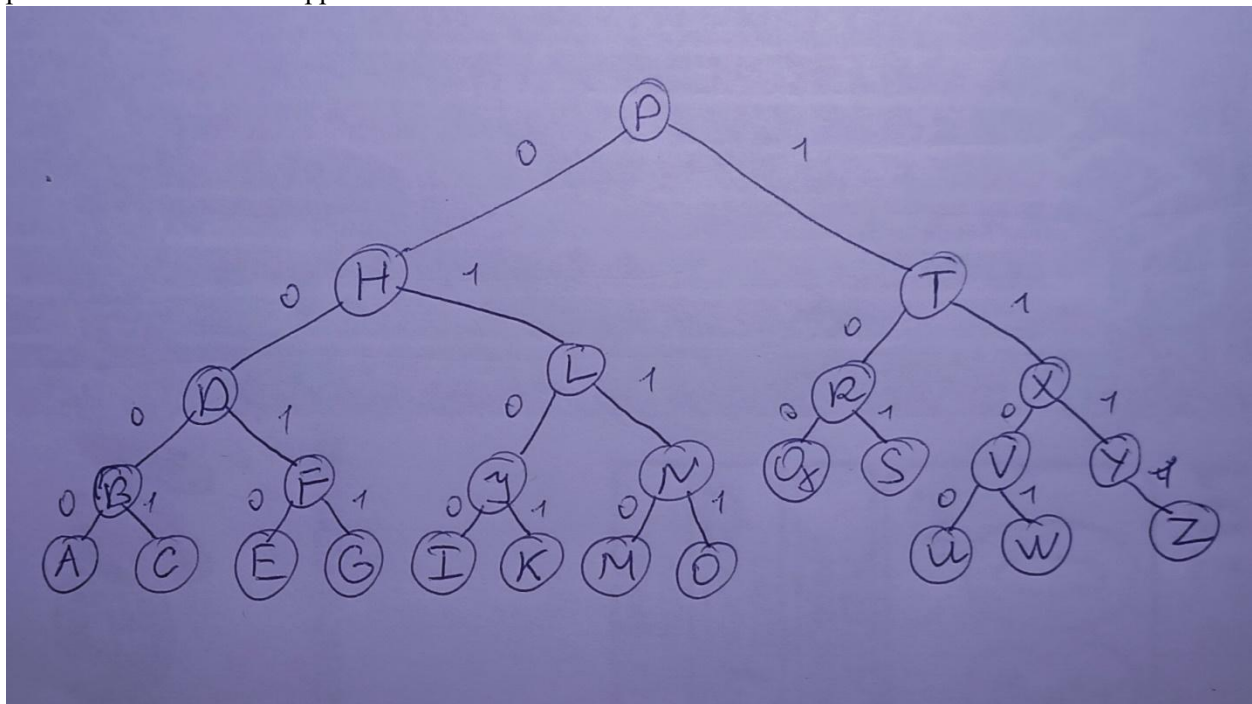
- 1)age of the user
- 2)location of the user
- 3)preferred age of partner
- 4)hobbies
- 5) ID of the user (will be generated by the system after registration)

Create two sets, "User" this is our user and "Target" this is the user we are looking for,

User = {age, location, hobbies}

Target = {preferred age of partner, location, hobbies}

After that, in each set, transfer all the characters in the string to the uppercase and turn them into binary value using binary tree, then turn this binary value to decimal value, then hash every one of them using DJB2 hash function, every hobby will be hashed separately. Then we will combine all the hashed values into one like $1 + 2 = 12$, then hash it again. The final hashed value will then be used to search for potential partners. The linked list approach will be used to handle collisions.



AVL tree of an alphabet based on ASCII value, right side is 1's, left side 0's

Example:

Age: 18

Location: Tashkent

Preferred age of a partner: 17

Hobbies: gym, books

1-> turn all strings to upper case

User: {18, TASHKENT, GYM, BOOKS}

Target: {17, TASHKENT, GYM, BOOKS}

2 -> turn into binary values

User: {18, 100001010010100100111, 00111110110, 000011101110101101}

Target: {17, 100001010010100100111, 00111110110, 000011101110101101}

3 -> turn the binary to decimal

User: {18, 1090855, 502, 15277}

Target: {17, 1090855, 502, 15277}

4 -> hash using djb2Hash

User: {5861582, 868194481, 193436348, 194223339}

Target: {5861581, 868194481, 193436348, 194223339}

5 -> merge all the values and hash again

User: {18686}

Target: {74249}

After this we will put the user's id to the value of {1868607238} in the hash table, and handle collisions with separate chaining.

Now we need to look for other user's account to suggest to our user, and here the separate chaining will be handy, as we will just in the hash table for {742410309}, and then traverse the linked list with various user ids in order to suggest every one of them to our user.

Code

```
main.java

import java.util.Scanner;

public class Main {

    public static int djb2Hash(String key) {
        int hashValue = 5381;

        for (char c : key.toCharArray()) {
            hashValue = (hashValue << 5) + hashValue + c;
        }

        return hashValue;
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        while(true) {
            String key = input.nextLine();
            int hash = djb2Hash(key);
            System.out.println("Hash value for '" + key + "': " + hash);
        }
    }
}
```

Code of djb2Hash in order to hash the values.

```
hashTable.java

public class CustomHashTable {

    public static class HashTable {
        private LinkedList<Integer>[] table;

        public HashTable(int capacity) {
            this.table = new LinkedList[capacity];
            for (int i = 0; i < capacity; i++) {
                table[i] = new LinkedList<>();
            }
        }

        public void put(int address, int value) {
            table[index].add(value);
        }

        public LinkedList<Integer> getValues(int address) {
            int index = address % table.length;
            return table[index];
        }

        public void printTable() {
            for (int i = 0; i < table.length; i++) {
                System.out.print("Index " + i + ": ");
                for (int value : table[i]) {
                    System.out.print(value + " ");
                }
                System.out.println();
            }
        }
    }
}
```

Code of the hash table

```

Main.java
public class HashingExample {
    public static int djb2Hash(String s) {
        int hash = 5381;
        for (char c : s.toCharArray()) {
            hash = ((hash << 5) + hash) + c; // hash * 33 + c
        }
        return hash;
    }
    public static int[] convertStringToBinary(String s) {
        int[] binaryValues = new int[s.length() * 8];
        int index = 0;
        for (char c : s.toCharArray()) {
            String binaryString = String.format("%8s", Integer.toBinaryString(c)).replace(' ', '0');
            for (char bit : binaryString.toCharArray()) {
                binaryValues[index++] = Character.getNumericValue(bit);
            }
        }
        return binaryValues;
    }
    public static int convertBinaryToDecimal(int[] binaryValues) {
        int decimal = 0;
        int power = binaryValues.length - 1;
        for (int bit : binaryValues) {
            decimal += bit * Math.pow(2, power);
            power--;
        }
        return decimal;
    }
    public static int hashAttribute(String attribute) {
        attribute = attribute.toUpperCase();
        int[] binaryValues = convertStringToBinary(attribute);
        int decimalValue = convertBinaryToDecimal(binaryValues);
        return djb2Hash(Integer.toString(decimalValue));
    }
    public static int hashHobbies(String[] hobbies) {
        int combinedHash = 0;
        for (String hobby : hobbies) {
            combinedHash += hashAttribute(hobby);
        }
        return combinedHash;
    }
    public static int hashCombinedValues(int age, int locationHash, int hobbiesHash) {
        return djb2Hash(Integer.toString(age + locationHash + hobbiesHash));
    }
}

```

```

Main.java
public class HashingExample {
    public static void main(String[] args) {
        int userAge = 25;
        String userLocation = "New York";
        String[] userHobbies = {"Reading", "Traveling"};
        int targetAgePreference = 24;
        String targetLocation = "Los Angeles";
        String[] targetHobbies = {"Traveling", "Photography"};
        int userLocationHash = hashAttribute(userLocation);
        int targetLocationHash = hashAttribute(targetLocation);
        int userHobbiesHash = hashHobbies(userHobbies);
        int targetHobbiesHash = hashHobbies(targetHobbies);
        int combinedUserHash = hashCombinedValues(userAge, userLocationHash, userHobbiesHash);
        int combinedTargetHash = hashCombinedValues(targetAgePreference, targetLocationHash, targetHobbiesHash);
        int finalUserHash = djb2Hash(Integer.toString(combinedUserHash));
        int finalTargetHash = djb2Hash(Integer.toString(combinedTargetHash));
        hashTable.put(userId, finalUserHash);
        int addressToGet = finalTargetHash;
        LinkedList<Integer> valuesAtAddress = hashTable.getValues(addressToGet);
    }
}

```

Main function

Limitations

1. **User Choices:** To avoid confusion, we'll provide a list of activities, hobbies, and cities for users to pick from. However, this might limit the options as we can't list every single hobby or city in the world. So, a small number of users might not find what they're looking for.
2. **Hash Function Importance:** Our recommendation system relies heavily on our hash function. If it doesn't work well, the system might give poor recommendations.
3. **Handling Collisions:** We plan to use separate chaining to deal with collisions in our hash table. But, we need to make sure this method remains efficient, especially when our hash table gets bigger. Long linked lists could slow down our system.
4. **Resizing:** The size of our hash table could become a problem. If we get more users and preferences, we might need to resize our hash table dynamically. This will help us maintain good performance.
5. **User Preferences:** A good recommendation system needs to accurately capture user preferences. If we oversimplify these preferences, or if they don't reflect the real interests of users, our recommendations might not be accurate.
6. **Scalability:** As we get more users and profiles, our recommendation system needs to be efficient and scalable. We need to make sure our system can handle a large number of users without slowing down.

Subject importance

1. **Efficient Data Access:** Hash tables help us store and find data quickly. This is very important in recommendation systems where we need to access user preferences and profiles fast.
2. **Better Search and Matching:** Recommendation systems use algorithms to match user preferences with potential items or profiles. Hash tables can make these algorithms more efficient and accurate.
3. **Handling Large Data:** Our recommendation systems deal with lots of user profiles and preferences. Hash tables help our system handle this large amount of data without slowing down.
4. **Personalized User Experience:** Quick access to user preferences allows for a more personalized user experience. With hash tables, we can make suggestions based on individual user profiles, leading to happier users.
5. **Real-world Use:** The skills gained from studying this subject can be used in real life. Recommendation systems are used in online shopping, content streaming platforms, social networks, and more to improve user experience and engagement.
6. **Algorithmic Thinking:** Designing and implementing recommendation systems involves algorithmic thinking. Studying this subject helps improve these fundamental computer science skills.
7. **Data Security and Privacy:** Understanding how to handle user data in recommendation systems is crucial for maintaining security and privacy. Hashing functions, which are part of hash tables, can help secure sensitive user information.
8. **Use in Other Fields:** The principles learned in building recommendation systems can be used in other fields such as data science, machine learning, and artificial intelligence.
9. **Continuous Learning and Adaptation:** Recommendation systems need to continuously learn and adapt. Studying this subject equips us with skills to adapt to changing user preferences, incorporate feedback, and improve our recommendation algorithm over time.